

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

За седьмой семестр

По дисциплине: «Обработка изображений в интеллектуальных системах»

Тема: «Обучение классификаторов средствами библиотеки PyTorch»

Выполнил:

Студент 4 курса
Группы ИИ-26(1)
Прокопюк А.Д.

Проверила:

Андренко К.В.

Брест 2026

Цель: научиться конструировать нейросетевые классификаторы и выполнять их обучение на известных выборках компьютерного зрения

Общее задание

1. Выполнить конструирование своей модели СНС, обучить ее на выборке по заданию (использовать **torchvision.datasets**). Предпочтение отдавать как можно более простым архитектурам, базирующимся на базовых типах слоев (сверточный, полносвязный, подвыборочный, слой нелинейного преобразования). Оценить эффективность обучения на тестовой выборке, построить график изменения ошибки (matplotlib);
2. Ознакомьтесь с state-of-the-art результатами для предлагаемых выборок (из материалов в сети Интернет). Сделать выводы о результатах обучения СНС из п. 1;
3. Реализовать визуализацию работы СНС из пункта 1 (выбор и подачу на архитектуру произвольного изображения с выводом результата);
4. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

Вариант 12

№ варианта	Выборка	Размер исходного изображения	Оптимизатор
12	Fashion-MNIST	28X28	Adadelata

Код программы:

main.py

```
import matplotlib.pyplot as plt
import torch
from torch import nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets
from torchvision.transforms import v2

from model import NeuralNetwork
from train import train_loop, test_loop

def plot_metrics(history):
    epochs = range(1, len(history["train_loss"]) + 1)

    plt.figure(figsize=(12,5))

    plt.subplot(1, 2, 1)
    plt.plot(epochs, history["train_loss"], "o-", label="Train Loss")
    plt.plot(epochs, history["test_loss"], "o-", label="Test Loss")
```

```

plt.title("Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(
    epochs, history["test_acc"], "s-", color="green", label="Test Accuracy"
)
plt.title("Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy (%)")
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.savefig("learning_curves.png", dpi=300)
print("Saved in 'learning_curves.png'")
plt.show()

if __name__ == "__main__":
    device = (
        torch.accelerator.current_accelerator().type
        if torch.accelerator.is_available()
        else "cpu"
    )
    print(f"Using {device} device")

    transforms = v2.Compose([
        v2.ToImage(),
        v2.ToDtype(torch.float32, scale=True)
    ])

    train_data = datasets.FashionMNIST(
        root="data", train=True, download=True, transform=transforms
    )
    test_data = datasets.FashionMNIST(
        root="data", train=False, download=True, transform=transforms
    )

    train_loader = DataLoader(train_data, batch_size=64, shuffle=True)
    test_loader = DataLoader(test_data, batch_size=64, shuffle=False)

    model = NeuralNetwork().to(device)
    loss_fn = nn.CrossEntropyLoss()
    optimizer = optim.Adadelta(model.parameters(), lr=1.0)

    history = {"train_loss": [], "test_loss": [], "test_acc": []}

    epochs = 5

```

```

for t in range(epochs):
    print(f"Epoch {t+1}\n")
    train_loss = train_loop(train_loader, model, loss_fn, optimizer, device)
    test_loss, test_acc = test_loop(test_loader, model, loss_fn, device)

    history["train_loss"].append(train_loss)
    history["test_loss"].append(test_loss)
    history["test_acc"].append(test_acc)

torch.save(model.state_dict(), "fashion_cnn.pth")
print("Saved PyTorch Model State to fashion_cnn.pth")

plot_metrics(history)

```

train.py

```

import torch

def train_loop(dataloader, model, loss_fn, optimizer, device):
    model.train()
    size = len(dataloader.dataset)
    num_batches = len(dataloader)
    train_loss = 0.0

    for batch, (X, y) in enumerate(dataloader):
        X, y = X.to(device), y.to(device)

        pred = model(X)
        loss = loss_fn(pred, y)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        train_loss += loss.item()

        if batch % 100 == 0:
            loss_val = loss.item()
            current = batch * dataloader.batch_size + len(X)
            print(f"loss: {loss_val:>7f} [{current:>5d}/{size:>5d}]")

    avg_train_loss = train_loss / num_batches
    return avg_train_loss

def test_loop(dataloader, model, loss_fn, device):
    model.eval()
    size = len(dataloader.dataset)
    num_batches = len(dataloader)
    test_loss, correct = 0.0, 0.0

```

```

with torch.no_grad():
    for X, y in dataloader:
        X, y = X.to(device), y.to(device)
        pred = model(X)
        test_loss += loss_fn(pred, y).item()
        correct += (pred.argmax(1) == y).type(torch.float).sum().item()

test_loss /= num_batches
correct /= size
print(
    f"Test Error:\n Accuracy: {(100*correct):>0.01f}%, Avg loss:
{test_loss:>8f}\n"
)

return test_loss, correct*100

```

model.py

```

import torch
from torch import nn

class NeuralNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            #1*28*28
            nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1),
            nn.ReLU(),
            #32*28*28
            nn.MaxPool2d(kernel_size=2, stride=2),
            #32*14*14
            nn.Conv2d(in_channels=32, out_channels= 64, kernel_size=3,
padding=1),
            #64*14*14
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),
            #64*7*7
        )

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64*7*7, 128),
            nn.ReLU(),
            nn.Linear(128, 10),
        )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x

```

```

if __name__ == "__main__":
    model = NeuralNetwork()
    input = torch.randn(64, 1, 28, 28)
    output = model(input)

    print(f"Input: {input.shape}")
    print(f"Output: {output.shape}")

```

predict.py

```

import random
import matplotlib.pyplot as plt
import torch
from torchvision import datasets
from torchvision.transforms import v2

from model import NeuralNetwork

CLASSES = [
    "T-shirt/top",
    "Trouser",
    "Pullover",
    "Dress",
    "Coat",
    "Sandal",
    "Shirt",
    "Sneaker",
    "Bag",
    "Ankle boot",
]

def predict_random_sample():
    device = (
        torch.accelerator.current_accelerator().type
        if torch.accelerator.is_available()
        else "cpu"
    )

    model = NeuralNetwork().to(device)
    model.load_state_dict(torch.load("fashion_cnn.pth", map_location=device))
    model.eval()

    transforms = v2.Compose(
        [v2.ToImage(), v2.ToDtype(torch.float32, scale=True)]
    )
    test_data = datasets.FashionMNIST(
        root="data", train=False, download=True, transform=transforms
    )

    idx = random.randint(0, len(test_data) - 1)

```

```

image, true_label = test_data[idx]

input_tensor = image.unsqueeze(0).to(device)
with torch.no_grad():
    output = model(input_tensor)
    pred_label = output.argmax(1).item()

print(f"Real class: {CLASSES[true_label]}")
print(f"Predicted class: {CLASSES[pred_label]}")

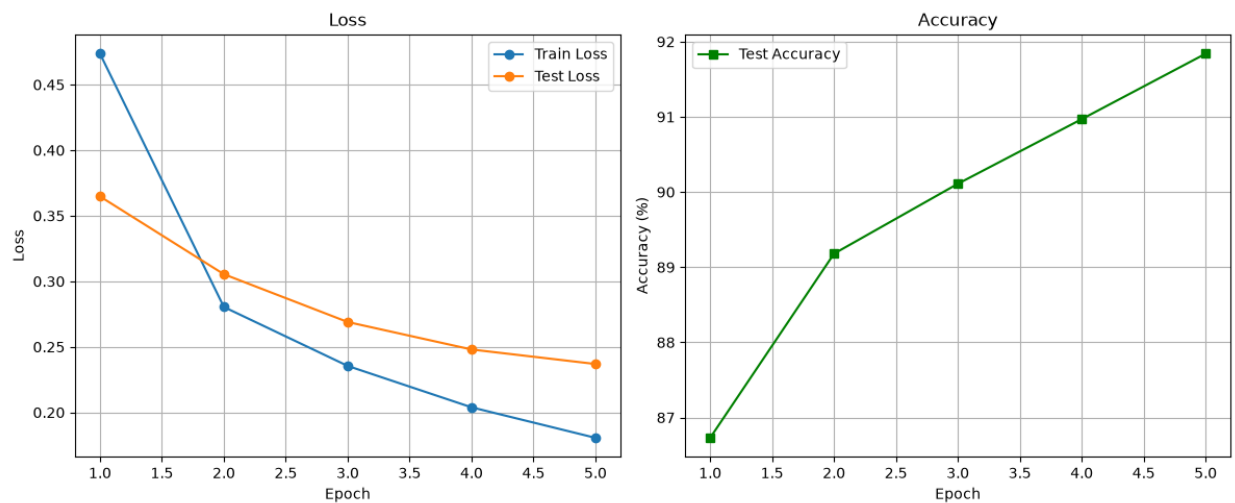
plt.imshow(image.squeeze(), cmap="gray")
plt.title(f"True: {CLASSES[true_label]}\nPred: {CLASSES[pred_label]}")
plt.axis("off")
plt.show()

if __name__ == "__main__":
    predict_random_sample()

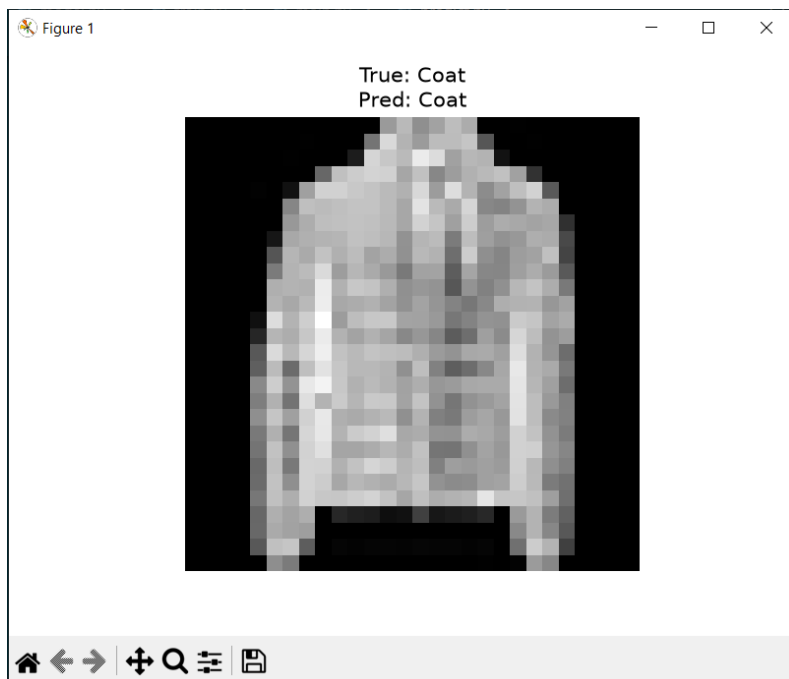
```

Результат:

Обучение:



Распознавание:



Вывод: научился конструировать нейросетевые классификаторы и выполнять их обучение на известных выборках компьютерного зрения